

Reemplazo de Imagen de Proceso con la Familia `exec()`

Malak Sanchez

Monitorias Sistemas Operativos

22 de septiembre de 2026

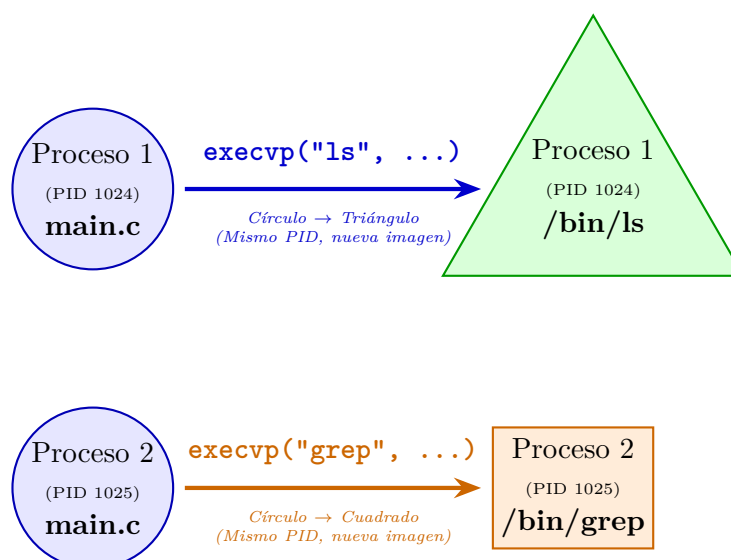
1. Motivación

La función `fork()` nos permite duplicar un proceso existente, pero ambos procesos continúan ejecutando el mismo código fuente. ¿Qué ocurre si deseamos que el nuevo proceso ejecute un programa completamente diferente (como `ls`, `grep` o un binario compilado de C)? Para este propósito, los sistemas POSIX proveen la familia de funciones `exec()`.

2. Idea Fundamental

Las llamadas al sistema de la familia `exec()` no crean un nuevo proceso. En su lugar, **reemplazan la imagen de memoria del proceso actual** por un nuevo programa ejecutable. El identificador de proceso (PID) se mantiene exactamente igual, pero el segmento de código, datos, pila y heap son destruidos y sustituidos por los del nuevo programa.

3. Transformación de la Imagen de Memoria



4. Variantes de la Familia exec()

Existen diferentes funciones según cómo reciben la ruta del ejecutable y los argumentos:

- **l (lista):** Los argumentos se pasan como una lista de parámetros terminada en NULL (ej. `execl`, `execlp`).
- **v (vector):** Los argumentos se pasan en un arreglo (*vector*) de cadenas terminado en NULL (ej. `execv`, `execvp`).
- **p (PATH):** Busca el ejecutable en las rutas definidas en la variable de entorno PATH (ej. `execlp`, `execvp`).
- **e (entorno):** Permite pasar un arreglo personalizado de variables de entorno (ej. `execve`).

5. Sintaxis Básica

```
#include <unistd.h>

// Uso con vector y búsqueda en PATH:
char *args[] = {"ls", "-l", "/tmp", NULL};
execvp("ls", args);

// Uso con lista:
execlp("ls", "ls", "-l", "/tmp", (char *)NULL);
```

6. Ejemplo Mínimo Funcional

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <unistd.h>
4  #include <sys/wait.h>
5
6  int main() {
7      pid_t pid = fork();
8
9      if (pid < 0) {
10         perror("fork failed");
11         return 1;
12     } else if (pid == 0) {
13         // Código del Hijo: Reemplazar imagen por '/bin/ls'
14         printf("[HIJO PID %d] Ejecutando execvp...\n", getpid());
15         char *args[] = {"ls", "-la", NULL};
16         execvp("ls", args);
17
18         // Si execvp tiene éxito, ESTA LINEA NUNCA SE EJECUTA
19         perror("execvp failed");
20         exit(1);
21     } else {
```

```
22      // Código del Padre: Esperar a que el hijo termine
23      printf("[PADRE PID %d] Esperando al hijo (PID %d)...\n",
              getpid(), pid);
24      wait(NULL);
25      printf("[PADRE PID %d] El hijo finalizó exitosamente.\n",
              getpid());
26  }
27
28  return 0;
29 }
```

7. Explicación Paso a Paso

1. `fork()`: Crea un proceso hijo duplicando la memoria del padre.
2. `execvp("ls", args)`: En el proceso hijo, sustituye su espacio de direcciones por la imagen binaria del comando `ls`.
3. **Punto de no retorno**: Si la llamada a `execvp` es exitosa, el código del programa original tras la línea del `execvp` desaparece por completo y el control se entrega a la función `main` del comando ejecutado.
4. Si `execvp` falla (por ejemplo, si el comando no existe), retorna `-1` y se ejecuta la siguiente línea (`perror`).

8. Qué Ocurre en Memoria

Al invocar cualquier función de la familia `exec`:

- Los segmentos de Código (*Text*), Datos (*Data*), BSS, Pila (*Stack*) y Heap son destruidos.
- El Kernel lee el encabezado ELF del ejecutable especificado y carga los nuevos segmentos en memoria.
- El contador de programa (PC) se inicializa en el punto de entrada del nuevo ejecutable.
- **Propiedades preservadas**: El PID, el PPID, los descriptores de archivos abiertos (salvo que tengan la bandera `FD_CLOEXEC`) y las credenciales de usuario se mantienen intactos.

9. Patrones Comunes de Uso

El patrón clásico en sistemas operativos UNIX es la combinación `fork()` + `exec()` + `wait()`, el cual forma la base de la ejecución de comandos en cualquier emulador de Shell o terminal.

10. Errores Comunes

- **Olvidar terminar la lista/arreglo con `NULL`:** Tanto el vector de argumentos de `execv` como la lista de `exec1` requieren obligatoriamente un apuntador `NULL` al final. De lo contrario, causará acceso inválido a memoria (*Segmentation Fault*).
- **Escribir código después de `exec`:** Pensar que la función `exec` retornará al código fuente original tras completar el nuevo programa. Recordar que `exec` solo retorna si ocurre un error.

11. Buenas Prácticas

- Colocar siempre una llamada a `perror()` y `exit(EXIT_FAILURE)` inmediatamente después de la invocación a `exec()`, para gestionar adecuadamente posibles errores de carga.
- Usar `execvp` cuando se ejecuten comandos estándar del sistema instalados en las rutas habituales del sistema.

12. Ejercicios

1. Escriba un programa en C que cree dos procesos hijos usando `fork()`. Haga que el primero ejecute `pwd` y el segundo `uname -s` mediante la familia `exec`.
2. Investigue `execve` y escriba un programa que ejecute un binario sencillo con un entorno personalizado que contenga una variable `TURN0`.